

MessageShift: Fine-Grained Data Augmentation for Graph Neural Networks

Weigang Lu
weiganglu@ust.hk
The Hong Kong University of Science
and Technology
Department of Civil and
Environmental Engineering
Hong Kong SAR, China

Zheng Liang*
liangz@ust.hk
The Hong Kong University of Science
and Technology
Department of Civil and
Environmental Engineering
Hong Kong SAR, China

Yaming Yang
yym@xidian.edu.cn
Xidian University
School of Computer Science and
Technology
Xi'an, China

Ziyu Zheng
zhengziyu@stu.xidian.edu.cn
Xidian University
School of Computer Science and
Technology
Xi'an, China

Meng Yan
mengyan@stu.xidian.edu.cn
Xidian University
School of Computer Science and
Technology
Xi'an, China

Beilei Ling
lingbeilei@stu.xidian.edu.cn
Xidian University
School of Computer Science and
Technology
Xi'an, China

Ziyu Guan
zyguan@xidian.edu.cn
Xidian University
School of Computer Science and
Technology
Xi'an, China

Wei Zhao
ywzhao@mail.xidian.edu.cn
Xidian University
School of Computer Science and
Technology
Xi'an, China

Abstract

Graph Neural Networks (GNNs) have become the dominant paradigm for machine learning on relational data, yet they remain susceptible to overfitting and noise in graph structures. While data augmentation has proven effective for regularization across domains, existing graph methods operate at coarse levels, such as perturbing entire structures or mixing node features. These approaches are context-agnostic and do not target the core computational process of GNNs: message passing. We introduce MESSAGESHIFT, a novel fine-grained data augmentation paradigm that operates directly on the messages, the atomic units of information, as they flow through the GNN. The core idea is to apply a **contextual perturbation** to each message by shifting it towards or away from the center of its local neighborhood. This provides a rich regularization effect, capable of both smoothing noisy messages and sharpening distinctive ones. Extensive experiments on a wide range of benchmark datasets demonstrate that MESSAGESHIFT consistently outperforms strong baselines across multiple GNN backbones.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Networks** → **Network algorithms**.

*Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792426>

Keywords

Graph Neural Networks, Graph Augmentation, Semi-supervised Node Classification

ACM Reference Format:

Weigang Lu, Zheng Liang, Yaming Yang, Ziyu Zheng, Meng Yan, Beilei Ling, Ziyu Guan, and Wei Zhao. 2026. MessageShift: Fine-Grained Data Augmentation for Graph Neural Networks. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3774904.3792426>

1 Introduction

Graph Neural Networks (GNNs) have emerged as a powerful class of models for learning representations on graph-structured data, achieving state-of-the-art results across a wide range of applications, from social network analysis and recommendation systems to drug discovery and molecular chemistry. The success of GNN stems from their ability to iteratively aggregate information from local neighborhoods, effectively capturing both the features of nodes and the topology of the graph. However, this expressive power comes at the cost of a large number of parameters, making GNN, like other deep models, prone to overfitting, particularly in settings with limited labeled data or noisy graph structures. This challenge has spurred significant research into effective regularization techniques, among which data augmentation stands out as a universally powerful paradigm.

The current landscape of graph data augmentation primarily focuses on two levels of perturbation. The first is structural perturbation, which involves modifying the graph's topology by randomly adding or dropping edges and nodes [10, 11, 28, 43]. While effective,

these methods alter the foundational structure on which the GNN operates, which can be suboptimal if the original graph topology is informative. The second level is feature-level perturbation, which includes masking node features or applying Mixup-style interpolations between the representations of different nodes [21, 33] or entire graphs [3, 23, 27]. These techniques, while model-agnostic, are often disconnected from the GNN’s learning dynamics since they treat the graph’s components as static data points to be mixed.

In this work, we ask a fundamental question: *What if we could perform data augmentation directly within the message passing process itself?* We propose that a more principled and fine-grained approach to graph data augmentation is *not* to alter the nodes or the structure, but to augment the very information that flows across them. To this end, we introduce MESSAGESHIFT, a novel data augmentation framework designed to operate directly on the messages exchanged between nodes. The core concept is simple yet powerful: instead of mixing a message with another, it perturbs each message by slightly “shifting” it. The direction and magnitude of this shift are intelligently determined by the message’s relationship to its local neighborhood context, the vector pointing from the message to its local mean. This allows for a continuous and rich spectrum of augmentation, from smoothing out noisy messages by pulling them toward the local consensus, to sharpening distinctive ones by pushing them further away.

MESSAGESHIFT is designed from the ground up as a model-agnostic and locality-aware augmentation technique. Its operations are strictly confined within a node’s one-hop neighborhood, respecting the fundamental principle of GNN. More importantly, MESSAGESHIFT’s full potential is unlocked through its adaptive capabilities. We equip it with three key strategies that can be combined to form an intelligent and dynamic regularizer: (1) a selective strategy that applies augmentation only to high-uncertainty “boundary” regions of the graph, identified via neighborhood label entropy; (2) an asymmetric shift mechanism that can favor smoothing over sharpening; and (3) a curriculum strategy that anneals the augmentation strength as training progresses. This adaptability allows MESSAGESHIFT to focus its regularization effect precisely where and when it is most needed, maximizing performance gains while minimizing unnecessary information loss.

Our main contributions can be summarized as follows:

- We propose MESSAGESHIFT, a new paradigm for graph data augmentation that is model-agnostic, locality-aware, and operates directly at the message-passing level.
- We introduce a suite of adaptive strategies to transform MESSAGESHIFT into a powerful and intelligent regularizer.
- We conduct extensive experiments on widely-used benchmark datasets, demonstrating that MESSAGESHIFT consistently and robustly improves the generalization performance of a variety of standard GNN architectures.

2 Backgrounds

2.1 Graph Neural Networks

Graph Neural Networks (GNNs) [5, 14, 18, 19, 31, 35, 37–39] are the prevailing approach for learning on graph-structured data. Most

GNNs follow a message-passing scheme, where each node aggregates information from its neighbors to iteratively update representations. By stacking layers, GNNs jointly encode node features and graph structure, achieving strong performance on tasks such as node classification [9, 21, 22, 24] and graph classification [40]. However, practical deployment remains challenging. GNNs can overfit when supervision is limited, are sensitive to structural/feature noise that propagates through message passing, and often require substantial labeled data to generalize well. These issues motivate lightweight techniques that improve robustness and generalization without adding significant computational cost. A widely adopted direction is graph augmentation, which perturbs structure or features (or interpolates representations) to provide regularization, increase training diversity, and improve noise tolerance.

2.2 Graph Augmentation

Graph data augmentation is an important approach to improve the generalization and robustness of GNNs. Existing methods can be broadly categorized into three classes: **structure-level augmentation** and **feature-level augmentation**. In addition, a distinct line of research, **graph mixup augmentation**, that combines both aspects and extends the Mixup data augmentation paradigm to graph data. However, these methods often work at coarse levels relative to the GNN message-passing mechanism, limiting their ability to address noise and uncertainty in graph signals.

2.2.1 Structure-level Augmentation. Structure-level perturbation methods modify the graph topology by adding, removing, or altering edges or nodes to produce augmented graphs. Early and widely adopted methods such as DropEdge [28] randomly remove edges during training to alleviate over-smoothing and reduce overfitting effects. Similarly, DropNode [11] removes nodes to diversify graph structures. In a more general way, DropMessage [10] extends these techniques to a unified framework by dropping some messages. These interventions operate globally and apply uniformly, thus lacking fine-grained control to distinguish noisy or uncertain messages from informative ones. More recent approaches introduce learnable or heuristic edits, for example, GAUG [43] systematically add or remove edges to improve homophily and connectivity. The Aggregation Buffer method [20] and ADEdgeDrop [6] extend DropEdge by parameterizing the aggregation process to improve stability of training and representation quality. Despite these advances, all these methods focus on structural modifications at the node or edge level and do not directly regulate individual messages passed along edges during GNN computation. Other works exploit subgraph sampling or view creation strategies to generate perturbed graph versions. Techniques such as MVGRL [16] creates subgraph views for contrastive learning. Frameworks like GraphCL [41] also applies global or subgraph-level augmentations to learn invariant node or graph embeddings in a contrastive manner. While effective for representation learning, these methods transform large graph regions, potentially over-smoothing locally important variations and lack selectivity in focusing on uncertain boundary regions.

2.2.2 Feature-level Augmentation. Feature-level perturbation methods diversify the node or edge attribute space to promote generalization. Contrastive augmentation methods like GraphCL

apply random feature masking or shuffling to increase view diversity. However, these operate solely at the input level without adapting to how features dynamically interact during aggregation. More sophisticated approaches incorporate probabilistic models, like Gaussian mixture models, to sample new graph-level features and enhance distributional coverage, yet they lack granularity at the individual message level within propagation paths [1, 2]. Methods leveraging random walk or diffusion mechanisms diffuse node features across local neighborhoods, capturing broader spatial information, but do so at the neighborhood or subgraph scale rather than isolating specific noisy or uncertain messages [4, 34]. Overall, these feature-level perturbations generally disregard the critical interplay of features within the message-passing framework, missing an opportunity to adaptively smooth noisy signals or sharpen distinctive messages during propagation. Our work addresses this limitation by directly perturbing individual messages guided by neighborhood uncertainty, offering fine-grained and adaptive control over feature propagation.

2.2.3 Graph Mixup Augmentation. Mixup-based approaches form an important branch, especially for graph-level tasks [7, 8, 13, 15, 25–27]. In graph classification, the semantic integrity lies in the whole graph rather than individual nodes, which makes mixup at the graph level more straightforward. However, such strategies face challenges when directly applied to node-level tasks, where each node has a distinct neighborhood that must be preserved. To address this, several node-level mixup methods have been proposed. For example, [33] mixes node features in pairs and blends their aggregated representations at each GNN layer. GraphMixup [36] introduces a reinforcement mechanism to adaptively adjust the mixing scale, targeting class imbalance in node classification. GraphMix [32] performs mixup on hidden node representations before classification layers. NodeMixup [21] extends mixup to both inter- and intra-class settings for labeled and unlabeled nodes, tackling the under-reaching issue. iGraphMix [17] further considers both node features and connections, directly mixing substructures of labeled nodes in the input graph. AGMixup [23] proposes a subgraph-based mixup method with adaptive strategies, allowing more flexible mixup operation for graphs.

3 Our Method: MESSAGESHIFT

In this section, we provide a detailed exposition of our proposed MESSAGESHIFT data augmentation framework. We begin by reviewing the fundamentals of message passing in GNNs. Next, we elaborate on the core concept and mathematical formulation of MESSAGESHIFT. Finally, we introduce a series of adaptive strategies that enable MESSAGESHIFT to be intelligently applied to different regions of the graph and at various stages of training, and we discuss the theoretical underpinnings of our approach.

3.1 Preliminaries: Message Passing in GNNs

GNNs learn representations for each node in a graph by iteratively aggregating information from its neighbors. A GNN layer can be abstracted as a Message Passing process, which primarily consists of three steps: message computation, message aggregation, and node update.

For a node v in a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, its hidden representation at layer k , denote as $h_v^{(k)}$, is updated based on its representation from the previous layer, $h_v^{(k-1)}$, and messages from its neighborhood $\mathcal{N}(v)$. The message from a neighbor node $u \in \mathcal{N}(v)$, $m_{u \rightarrow v}^{(k)}$, is computed as:

$$m_{u \rightarrow v}^{(k)} = \text{MESSAGE}^{(k)}(h_u^{(k-1)}, h_v^{(k-1)}, e_{uv}), \quad (1)$$

where e_{uv} represents the features of the edge (u, v) . In many prominent GNN architectures, such as GCN [18], the message is primarily determined by the source node’s representation, i.e., $m_{u \rightarrow v}^{(k)} = \Phi(h_u^{(k-1)})$, where Φ is a learnable linear transformation.

3.2 MESSAGESHIFT: Augmenting the Message Flow

Existing augmentation techniques for GNNs primarily operate at the feature or structural level, such as masking node features or dropping edges. While effective, these methods are indirect: they modify the static inputs to the message passing mechanism, rather than the dynamic information flow itself. We argue that a more direct and fine-grained approach is to augment the messages—the atomic units of information—as they are being computed.

This raises a fundamental design question: *what constitutes a meaningful perturbation for a message?* A message is not an independent data point and its semantic value is defined by its local context. A simple approach, like adding unstructured random noise, is context-agnostic and risks corrupting the signal. Another common augmentation paradigm, interpolation (i.e., Mixup), is also ill-suited for this task. Mixing a message $m_{u \rightarrow v}$ with another arbitrary message would disregard the specific context of the target node v and could combine semantically unrelated information.

Therefore, we propose a new paradigm of contextual perturbation. The core idea of MESSAGESHIFT is *not to combine the message with external, context-agnostic information, but rather to apply an intelligent, directional “shift” to each message derived purely from its own local neighborhood context*. This process reframes augmentation from modifying static inputs to perturbing the computational flow, and it involves three key steps. This three-step process, which forms the core of the MESSAGESHIFT operation, is visualized in Figure 1. The figure provides an intuitive illustration of how a message is perturbed relative to its local context. It shows the initial definition of the neighborhood mean, the subsequent calculation of a context-aware direction vector, and the final shift that produces the augmented message.

3.2.1 Step 1: Calculating the Local Context. For a target node v , the set of all incoming messages from its neighbors, $\{m_{u \rightarrow v} | u \in \mathcal{N}(v)\}$, collectively defines its local information environment. We use the mean of these messages to represent the “center” of this local context, which we term the mean message, $\bar{m}_v$:

$$\bar{m}_v = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}. \quad (2)$$

This mean message, $\bar{m}_v$, captures the average characteristics of the information within node v ’s neighborhood.

3.2.2 Step 2: Defining the Shift Direction. For each specific message $m_{u \rightarrow v}$, we compute its difference from the neighborhood

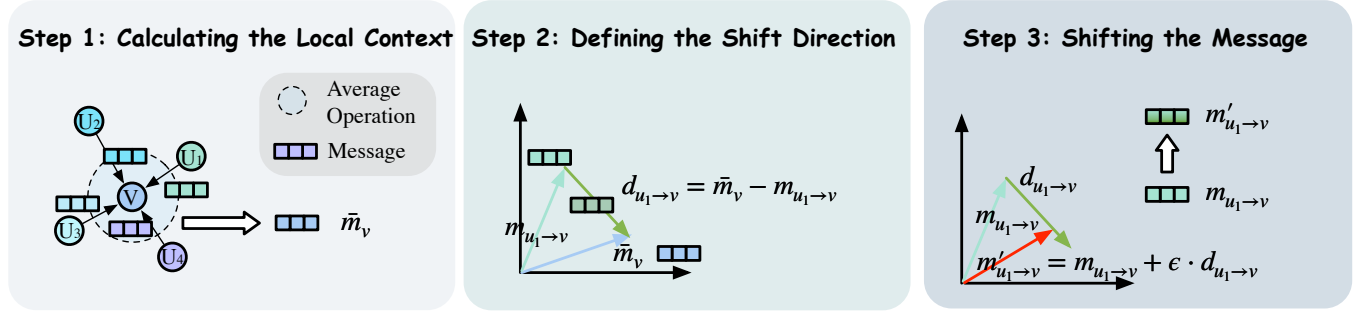


Figure 1: An illustration of the core MESSAGESHIFT operation in three steps. (Step 1) For a target node v , all incoming messages $\{m_{u_i \rightarrow v}\}$ from its neighborhood are first aggregated via an averaging operation to define the local context, represented by the mean message $\bar{m}_v$. (Step 2) A direction vector $d_{u_1 \rightarrow v}$ is then computed for a specific message $m_{u_1 \rightarrow v}$, pointing from the original message towards the local context center $\bar{m}_v$. (Step 3) Finally, the original message is shifted along this direction by a randomly scaled amount ($\epsilon d_{u_1 \rightarrow v}$), producing the augmented message $m'_{u_1 \rightarrow v}$. The visualization here depicts a smoothing effect ($\epsilon > 0$), where the message is pulled closer to the neighborhood mean.

center $\bar{m}_v$. This difference defines a direction vector $d_{u \rightarrow v}$ that points from the message towards the neighborhood center:

$$d_{u \rightarrow v} = \bar{m}_v - m_{u \rightarrow v}. \quad (3)$$

This vector represents the direction and magnitude required to “pull” the current message back towards the neighborhood average.

3.2.3 Step 3: Shifting the Message. We generate the augmented message $m'_{u \rightarrow v}$ by shifting the original message $m_{u \rightarrow v}$ along the direction vector $d_{u \rightarrow v}$ by a small amount, controlled by a random coefficient ϵ :

$$m'_{u \rightarrow v} = m_{u \rightarrow v} + \epsilon d_{u \rightarrow v}. \quad (4)$$

Here, ϵ is a random variable sampled from a distribution controlled by a hyperparameter $\alpha \in (0, 1]$, such as $\epsilon \sim \text{Uniform}(-\alpha, \alpha)$. This simple operation yields a rich set of augmentation effects: **(1) Message Smoothing:** When $\epsilon > 0$, the message is pulled towards the neighborhood mean. This can effectively suppress noise from anomalous neighbors, leading to a smoother message flow and acting as a form of regularization. **(2) Message Sharpening:** When $\epsilon < 0$, the message is pushed away from the neighborhood mean. This effectively amplifies the message’s uniqueness, which can help preserve node identity in highly homophilous neighborhoods and potentially mitigate over-smoothing.

3.3 Augmenting the Message Flow with Adaptive Strategies

The core MESSAGESHIFT operation described in Section 3.2 provides a powerful, universal mechanism for message augmentation. However, its full potential is unlocked when its application is adapted to the specific topological characteristics of the graph and the dynamics of the training process. A “one-size-fits-all” approach, while effective, can be further improved. In this section, we introduce three adaptive strategies that transform MESSAGESHIFT from a universal augmenter into an intelligent and precise regularization framework. These strategies answer three critical questions: where, how, and when to apply augmentation.

3.3.1 Where to Augment? Selective Targeting (ST). Not all nodes require augmentation equally and applying augmentation uniformly is suboptimal. In a homophilous “core region” where neighbors are highly class-consistent, the message flow is already clean, and excessive augmentation could introduce unnecessary noise. Conversely, in a heterophilous “boundary region” with mixed neighbor classes, the model is more prone to confusion, making it the ideal location for regularization to improve robustness.

To differentiate these regions, we use Neighborhood Label Entropy to quantify the uncertainty of a node’s local environment. For a node v , its neighborhood label entropy is defined as:

$$H(\mathcal{Y}_{N(v)}) = - \sum_{c=1}^C P_v(c) \log P_v(c), \quad (5)$$

where C is the total number of classes, and $P_v(c) = \frac{|\{u \in N(v) | y_u = c\}|}{N(v)}$ is the empirical probability of class c in the neighborhood. We pre-compute this value for all nodes and set an entropy threshold H_τ (e.g., the median entropy across the graph). During training, MESSAGESHIFT is then selectively applied only to messages targeting nodes where $H(\mathcal{Y}_{N(v)}) > H_\tau$.

3.3.2 How to Augment? Asymmetric Perturbation (AP). The optimal balance between smoothing and sharpening is not a global property of the graph, but rather a local one. A neighborhood with highly coherent features (strong homophily) is susceptible to over-smoothing, and its messages would benefit from sharpening to preserve identity. Conversely, a neighborhood with diverse features (heterophily or noise) requires smoothing to denoise and extract a more robust signal. A fixed augmentation strategy fails to capture this local dynamic.

We dynamically adjust the augmentation bias based on the Local Feature Coherence of each neighborhood. We define this coherence for a node v as the average cosine similarity between its own message representation and those of its neighbors before augmentation:

$$\text{coh}(v) = \text{avg}_{u \in N(v)} \left(\frac{\mathbf{m}_{v \rightarrow} \cdot \mathbf{m}_{u \rightarrow v}}{\|\mathbf{m}_{v \rightarrow}\| \|\mathbf{m}_{u \rightarrow v}\|} \right), \quad (6)$$

where $\mathbf{m}_{v \rightarrow \cdot}$ is a proxy for node v 's own message (e.g., its transformed feature vector). We normalize this score to a range of $[0, 1]$, let's call it $\text{coh}'(v)$. This coherence score then directly controls the asymmetry of the shift for all messages targeting node v . We define the bounds of the sampling interval for ϵ_v as:

$$\beta_{pos} = \text{coh}'(v) \quad \text{and} \quad \beta_{neg} = -(1 - \text{coh}'(v)). \quad (7)$$

The shift coefficient is then sampled as $\epsilon_v \sim \text{Uniform}(\alpha\beta_{neg}, \alpha\beta_{pos})$. This creates a dynamic feedback loop: **(1) High Coherence**, when $\text{coh}'(v) \rightarrow 1$, the sampling range (e.g., $[-0.1\alpha, 0.9\alpha]$) strongly favors sharpening, pushing messages away from the already-cohesive center to prevent over-smoothing. **(2) High Coherence**, when $\text{coh}'(v) \rightarrow 0$, the sampling range (e.g., $[-0.9\alpha, 0.1\alpha]$) strongly favors smoothing, pulling diverse messages towards a stable center to denoise and regularize. This approach eliminates the need for manual tuning of the augmentation bias, allowing the strategy to adapt intelligently to the unique local structure of each node.

3.3.3 When to Augment? Information Flow Regulation (IFR).

The ideal moment to apply strong regularization is when the model is most uncertain. In a GNN, this uncertainty is particularly consequential. A node with a high predictive loss is not merely a misclassified sample and it is a potential source of corrupted information for its entire neighborhood in subsequent layers. We therefore reframe the question of "when to augment" from a matter of training time to a matter of model need, proposing a proactive strategy to regulate the quality of information at these critical, unstable nodes.

Our Information Flow Regulation directly links augmentation strength to model uncertainty. The per-node strength α_v for messages targeting node v is dynamically modulated by its real-time predictive loss, $\mathcal{L}_v$:

$$\alpha_v = \alpha_{base} I(\mathcal{L}_v), \quad (8)$$

where α_{base} is a global strength parameter and $I(\cdot)$ is a normalized intensity function, i.e., $I(\mathcal{L}_v) = \text{clamp}(\frac{\mathcal{L}_v}{\log C}, 0, 1)$. Here, $\log C$ is the maximum value of the classification loss.

This creates a powerful feedback loop that influences training in two ways. When the model struggles with node v (high $\mathcal{L}_v$), the corresponding α_v increases. This intensifies the MESSAGESHIFT perturbation on its incoming messages. By confronting the model with a more challenging and varied set of inputs for the struggling node, this strong perturbation acts as a form of online hard example mining in the message space. To minimize its loss under this heightened uncertainty, the model is compelled to discard superficial or noisy features and instead learn to rely on the most salient and invariant patterns within the neighborhood's information, inherently leading to a more robust representation.

3.4 Time Complexity

In this section, we analyze the computational complexity of the proposed MESSAGESHIFT and show that it preserves the asymptotic complexity of the GNN backbone.

For a standard message-passing GNN layer on a graph with $|E|$ edges and feature dimension D , the dominant cost comes from edge-wise message aggregation, which is $O(|E|D)$. Within each layer, MESSAGESHIFT performs only lightweight edge-wise operations: (1) computing a per-node mean of incoming messages via a single pass over edges, and (2) for each message, computing its deviation from

the mean and applying an element-wise shift. These steps are all linear in the number of edges and feature dimension, i.e., $O(|E|D)$.

Our adaptive strategies introduce minimal overhead. Selective targeting is pre-computed once before training and does not affect the training loop. Adaptive asymmetry adds an $O(D)$ cosine similarity per edge, totaling $O(|E|D)$. Regulating information flow maintains simple per-node statistics with $O(1)$ updates. Overall, the time complexity of MESSAGESHIFT remains $O(|E|D)$ per layer.

4 Experiment

Table 1: Datasets Statics.

Dataset	# Nodes	# Edges	# Features	# Classes
Cora	2,708	5,278	1,433	7
Citeseer	3,327	4,614	3,703	6
Pubmed	19,717	44,324	500	3
Amazon Photo	7,650	119,081	745	8
Amazon Computers	13,381	245,778	767	10
Coauthor CS	18,333	81,894	6,805	15
Coauthor Physics	34,493	247,962	8,415	5

In this section, we present a comprehensive set of experiments to evaluate the effectiveness of MESSAGESHIFT. We instantiate our method into two primary variants for evaluation:

- **MESSAGESHIFT_{Base}**: The core version of our method using context-aware message perturbation (cf. Section 3.2).
- **MESSAGESHIFT_{Ada}**: The fully adaptive version that incorporates all our proposed strategies (cf. Section 3.3).

Our goal is to answer the following key research questions:

- **RQ1 (Overall Performance)**: How does MESSAGESHIFT perform on the node classification task against a suite of strong baseline augmentation methods across various datasets and GNN backbones?
- **RQ2 (Component Efficacy)**: What is the individual contribution of each adaptive component (ST, AA, ALHGR) in MESSAGESHIFT?
- **RQ3 (Parameter Robustness)**: How sensitive is MessageShift's performance to its main hyperparameter?
- **RQ4 (Representation Quality)**: Does our MESSAGESHIFT's "sharpening-smoothing" capability effectively produce high-quality node representations?

4.1 Experimental Setup

4.1.1 Hardware and Software. MESSAGESHIFT is implemented based on the Torch Geometric library [12] and PyTorch 3.7.1 with Intel(R) Core(TM) i9-10980XE CPU @ 3.00GHz and one NVIDIA A100 GPUs with 40GB memory.

4.1.2 Datasets. We assess the performance of MESSAGESHIFT using seven widely-adopted real-world graph datasets: Cora, Citeseer, Pubmed [29], Coauthor Physics, Coauthor CS, Amazon Photo, and Amazon Computers [30]. These datasets exhibit diversity in node features, graph structures, and task complexities, offering a comprehensive benchmark for evaluating the generalizability of our approach. The details of all the graphs are provided in Table 1.

Table 2: Node Classification Accuracy $\pm$ Standard Deviation (%) on All Graphs.

GNN Arch. +Aug. Methods	Cora	Citeseer	Pubmed	CS	Physics	Photo	Computers	<i>Impro.</i>
GCN	80.88 $\pm$ 0.31	68.70 $\pm$ 0.73	78.30 $\pm$ 0.33	91.62 $\pm$ 0.63	93.09 $\pm$ 0.59	91.74 $\pm$ 0.65	88.80 $\pm$ 0.61	0.00%
+DROPEdge	81.24 $\pm$ 0.19	68.58 $\pm$ 0.62	78.60 $\pm$ 0.34	92.28 $\pm$ 0.14	93.11 $\pm$ 0.82	92.03 $\pm$ 0.28	88.51 $\pm$ 0.31	+0.20%
+DROPNode	80.38 $\pm$ 0.83	66.69 $\pm$ 0.94	75.86 $\pm$ 0.51	91.74 $\pm$ 0.26	92.85 $\pm$ 0.33	91.89 $\pm$ 0.20	87.68 $\pm$ 0.40	-1.13%
+ATTRMask	80.86 $\pm$ 0.71	68.50 $\pm$ 0.70	78.58 $\pm$ 0.62	92.17 $\pm$ 0.19	93.18 $\pm$ 0.82	92.21 $\pm$ 0.13	88.85 $\pm$ 0.43	+0.19%
+PAIRNorm	81.30 $\pm$ 0.73	68.64 $\pm$ 0.92	78.64 $\pm$ 0.51	92.04 $\pm$ 0.17	93.45 $\pm$ 0.33	92.36 $\pm$ 0.27	89.16 $\pm$ 0.15	+0.40%
+NODEMixUP	81.45 $\pm$ 0.92	69.04 $\pm$ 0.52	78.82 $\pm$ 0.64	92.38 $\pm$ 0.32	93.58 $\pm$ 0.26	92.90 $\pm$ 0.14	89.27 $\pm$ 0.41	+0.72%
+AGMIXUP	82.01 $\pm$ 0.41	69.42 $\pm$ 1.01	79.37 $\pm$ 0.61	92.67 $\pm$ 0.71	93.85 $\pm$ 0.92	93.02 $\pm$ 0.17	89.20 $\pm$ 0.40	+1.09%
+MESSAGESHIFT _{Base}	81.45 $\pm$ 0.53	69.97 $\pm$ 0.24	79.03 $\pm$ 0.25	92.74 $\pm$ 0.09	93.92 $\pm$ 0.14	93.19 $\pm$ 0.05	89.90 $\pm$ 0.13	+1.20%
+MESSAGESHIFT _{Ada}	82.19$\pm$0.39	70.30$\pm$0.16	80.06$\pm$0.47	93.58$\pm$0.05	94.57$\pm$0.01	93.47$\pm$0.06	90.06$\pm$0.02	+1.89%
GAT	80.74 $\pm$ 0.86	68.78 $\pm$ 0.27	77.40 $\pm$ 0.81	91.95 $\pm$ 0.19	92.97 $\pm$ 0.10	92.10 $\pm$ 0.21	88.56 $\pm$ 0.24	0.00%
+DROPEdge	80.68 $\pm$ 0.81	68.62 $\pm$ 0.46	77.18 $\pm$ 0.64	92.27 $\pm$ 0.14	93.07 $\pm$ 0.06	92.76 $\pm$ 0.25	88.66 $\pm$ 0.47	+0.10%
+DROPNode	77.63 $\pm$ 1.98	65.42 $\pm$ 1.62	76.32 $\pm$ 1.32	91.25 $\pm$ 0.32	92.37 $\pm$ 0.12	90.68 $\pm$ 0.57	86.48 $\pm$ 0.33	-2.20%
+ATTRMask	80.60 $\pm$ 1.03	68.58 $\pm$ 0.67	77.53 $\pm$ 0.31	92.04 $\pm$ 0.07	93.12 $\pm$ 0.22	92.36 $\pm$ 0.72	88.31 $\pm$ 0.61	-0.01%
+PAIRNorm	80.83 $\pm$ 0.81	68.98 $\pm$ 0.42	77.60 $\pm$ 0.22	92.74 $\pm$ 0.16	93.27 $\pm$ 0.24	92.51 $\pm$ 0.84	88.71 $\pm$ 0.36	+0.35%
+NODEMixUP	81.02 $\pm$ 0.76	69.41 $\pm$ 0.50	77.87 $\pm$ 0.24	92.53 $\pm$ 0.26	93.56 $\pm$ 0.28	92.17 $\pm$ 0.53	89.04 $\pm$ 0.65	+0.54%
+AGMIXUP	81.28 $\pm$ 0.82	69.70 $\pm$ 0.28	79.42 $\pm$ 0.57	92.84 $\pm$ 0.23	93.87 $\pm$ 0.13	92.79 $\pm$ 0.23	89.20 $\pm$ 0.48	+1.15%
+MESSAGESHIFT _{Base}	81.27 $\pm$ 0.21	69.94 $\pm$ 1.02	79.28 $\pm$ 0.55	92.98 $\pm$ 0.11	94.26 $\pm$ 0.12	92.54 $\pm$ 0.39	89.46 $\pm$ 0.08	+1.25%
+MESSAGESHIFT _{Ada}	82.46$\pm$0.54	70.21$\pm$0.53	79.97$\pm$0.57	93.23$\pm$0.18	94.72$\pm$0.08	92.94$\pm$0.21	89.63$\pm$0.01	+1.85%
GraphSAGE	80.07 $\pm$ 0.19	67.28 $\pm$ 0.39	76.58 $\pm$ 0.50	92.27 $\pm$ 0.12	93.10 $\pm$ 0.21	92.26 $\pm$ 0.25	88.65 $\pm$ 0.44	0.00%
+DROPEdge	79.14 $\pm$ 0.90	67.68 $\pm$ 1.19	76.68 $\pm$ 0.75	93.34 $\pm$ 0.16	93.34 $\pm$ 0.12	93.01 $\pm$ 0.05	87.31 $\pm$ 0.15	+0.04%
+DROPNode	75.14 $\pm$ 1.50	64.01 $\pm$ 2.65	75.20 $\pm$ 1.33	90.78 $\pm$ 1.67	90.67 $\pm$ 0.32	90.51 $\pm$ 1.47	85.60 $\pm$ 1.35	-3.20%
+ATTRMask	78.54 $\pm$ 0.46	69.08 $\pm$ 2.04	76.14 $\pm$ 0.67	93.07 $\pm$ 0.14	93.01 $\pm$ 0.11	93.23 $\pm$ 0.59	87.88 $\pm$ 1.29	+0.16%
+PAIRNorm	80.47 $\pm$ 0.52	69.42 $\pm$ 0.89	77.26 $\pm$ 0.56	93.46 $\pm$ 0.10	93.76 $\pm$ 0.28	93.26 $\pm$ 0.14	87.96 $\pm$ 0.89	+0.98%
+NODEMixUP	80.72 $\pm$ 0.67	69.73 $\pm$ 0.46	77.68 $\pm$ 0.82	93.97 $\pm$ 0.28	93.65 $\pm$ 0.36	93.41 $\pm$ 0.48	88.65 $\pm$ 0.46	+1.37%
+AGMIXUP	80.91 $\pm$ 0.54	69.76 $\pm$ 0.41	78.06 $\pm$ 0.66	94.06 $\pm$ 0.22	93.92 $\pm$ 0.59	93.55 $\pm$ 0.56	88.34 $\pm$ 0.58	+1.51%
+MESSAGESHIFT _{Base}	81.17 $\pm$ 0.62	70.17 $\pm$ 0.48	78.47 $\pm$ 0.42	94.65 $\pm$ 0.18	94.36 $\pm$ 0.16	94.17 $\pm$ 0.53	89.45 $\pm$ 0.21	+2.15%
+MESSAGESHIFT _{Ada}	82.37$\pm$0.26	71.03$\pm$0.17	79.56$\pm$0.52	95.21$\pm$0.28	94.61$\pm$0.24	94.47$\pm$0.12	89.82$\pm$0.17	+2.98%

4.1.3 Backbone GNNs. For the backbone GNN models, we select three of the most representative architectures: GCN [18], GAT [31], and GraphSAGE [14]. Each GNN is implemented with a standard 2-layer structure to ensure a fair and consistent comparison.

4.1.4 Baseline Augmentation Methods. We compare our MESSAGESHIFT against a comprehensive set of baselines, including:

- **DROPEdge** [28]: Randomly removes a fraction of edges from the graph at each training epoch.
- **DROPNode** [11]: Randomly masks out a fraction of nodes (and their connections) from the computation.
- **ATTRMask** [41]: Randomly masks a fraction of node feature dimensions, setting them to zero.
- **PAIRNorm** [42]: A normalization layer applied to GNNs to prevent feature scaling issues and potentially mitigate over-smoothing.
- **NODEMixUP** [21]: A graph mixup method that chooses mixup pairs from different node sets.

- **AGMIXUP** [23]: An adaptive graph mixup method that mixes subgraphs.

4.1.5 Parameters. For a fair comparison, we perform a systematic hyperparameter search for all methods. The search space is defined as follows:

- **Learning Rate:** {0.001, 0.002, 0.005, 0.01, 0.02, 0.05}
- **Weight Decay Rate:** { $5e-4$, $5e-5$, $5e-7$, $5e-9$ }
- **hidden Dimensionality:** {64, 128, 256}
- **Dropout Rate:** {0.3, 0.5, $\dots$, 0.9}
- **Edge Dropping/Feature Masking/Node Dropping Rate:** {0.1, 0.2, $\dots$, 0.8}
- **Uniform Distribution Hyperparameter α :** {0.1, 0.5, 0.7}
- **Entropy Threshold H_r :** {0.5, 0.7}

4.1.6 Implementation Details. To ensure statistical significance, we report the mean accuracy and standard deviation over 10 runs

Table 3: Ablation Study.

Configuration	Cora	CS
<i>Baseline</i>		
Vanilla	80.88 $\pm$ 0.31	91.62 $\pm$ 0.63
<i>Core Mechanism Analysis</i>		
+ Smoothing Only ($\epsilon > 0$)	81.23 $\pm$ 0.27	91.86 $\pm$ 0.12
+ Sharpening Only ($\epsilon < 0$)	79.86 $\pm$ 0.84	90.58 $\pm$ 1.21
MESSAGESHIFT_{Base}	81.45 $\pm$ 0.53	92.74 $\pm$ 0.09
<i>Adaptive Strategy Analysis</i>		
MESSAGESHIFT _{Base} + ST	81.78 $\pm$ 0.42	93.26 $\pm$ 0.11
MESSAGESHIFT _{Base} + AP	82.04 $\pm$ 0.51	93.41 $\pm$ 0.08
MESSAGESHIFT _{Base} + IFR	81.65 $\pm$ 0.32	93.02 $\pm$ 0.09
<i>Full Model</i>		
MESSAGESHIFT_{Ada}	82.19$\pm$0.39	93.58$\pm$0.05

with different random seeds for each experimental setting. Hyperparameters for all baseline methods are carefully tuned on the validation set.

4.2 RQ1: Main Results on Node Classification

In this section, we evaluate the effectiveness of MESSAGESHIFT_{Base} and MESSAGESHIFT_{Ada}, against the baselines on the node classification task.

The results are summarized in Table 2. As shown, our MESSAGESHIFT framework consistently outperforms the vanilla backbones and standard augmentation techniques across almost all datasets and GNN architectures, including strong recent baselines like AGMIXUP and NODEMIXUP. For instance, when integrated with GCN, MESSAGESHIFT_{Ada} boosts accuracy by an impressive average of +1.89% over the vanilla GCN baseline, showcasing a substantial improvement. Similar robust gains are observed with GAT (+1.85%) and GraphSAGE (+2.98%), underscoring its broad applicability and effectiveness across different message-passing mechanisms. Even the fundamental MessageShift operation (MESSAGESHIFT_{Base}), which applies a uniform augmentation strategy without our explicit adaptive controls, proves highly competitive, generally surpassing or matching many traditional heuristic methods like PAIRNORM.

When compared with other augmentation baselines, the advantages of MESSAGESHIFT_{Ada} become even clearer. Traditional heuristics like DROPEdge, DROPNode, and ATTRMask give limited or inconsistent results, with DROPNode sometimes harming performance due to random perturbations. Interpolation-based methods such as NODEMIXUP and AGMIXUP provide stronger gains, but MESSAGESHIFT_{Ada} still performs better. This confirms our main argument: perturbing message flow directly, with adaptive guidance, is a more precise and effective way to regularize GNNs than static graph edits or feature mixing. The consistent gains across different architectures highlight that MESSAGESHIFT is architecture-independent, making it a general and powerful solution for improving graph representation learning.

4.3 RQ2: Ablation Study

To deconstruct the effectiveness of our framework, we conduct a two-stage ablation study. The first stage validates the core MESSAGESHIFT mechanism, while the second quantifies the contribution of each adaptive strategy.

4.3.1 Core Mechanism Analysis. We first evaluate the fundamental components of the message shifting idea. We compare the vanilla GCN against variants of our method that isolate the smoothing ($\epsilon > 0$) and sharpening ($\epsilon < 0$) effects, and compare them to the symmetric MESSAGESHIFT_{Base}. The results show a clear trend: on both Cora and CS, applying only smoothing provides a modest but consistent improvement over the vanilla baseline. In contrast, applying only sharpening degrades performance, suggesting that exclusively amplifying message distinctiveness can be detrimental. The key insight is that MESSAGESHIFT_{Base}, which symmetrically combines both smoothing and sharpening, significantly outperforms not only the vanilla model but also the one-sided variants. For instance, on CS, it achieves 92.74%, a gain of over 1.1 points over the baseline and nearly 0.9 points over smoothing alone. This confirms our central hypothesis: the most effective regularization comes from a balanced and dynamic interplay between reducing noise and emphasizing uniqueness.

4.3.2 Adaptive Strategy Analysis. Starting with the base version, we incrementally add each of the three adaptive strategies: Selective Targeting (ST), Asymmetric Perturbation (AP), and Information Flow Regulation (IFR). The results consistently show that all three components contribute positively. On both datasets, AP provides the largest single improvement, boosting accuracy to 82.04% on Cora and 93.41% on CS. This highlights the importance of dynamically tailoring the smoothing/sharpening bias to the local neighborhood’s characteristics. Finally, the full MESSAGESHIFT_{Ada} model, which integrates all components, achieves the highest accuracy in all cases (82.19% on Cora, 93.58% on CS). This demonstrates a clear synergistic effect, where the components work together to form a more powerful and intelligent regularization framework than any single part in isolation.

4.4 RQ3: Hyperparameter Analysis

We analyze the sensitivity of our model to the key hyperparameters governing the MESSAGESHIFT mechanism: the augmentation strength α and the entropy threshold H_τ . We conduct these tests on the Cora and CS datasets with the GCN backbone, varying one parameter while keeping others at their optimal values. The results are visualized in Fig. 2.

4.4.1 Sensitivity to Augmentation Strength α . The base strength α controls the overall magnitude of the message perturbation. As shown in the results, the model’s performance follows a clear and desirable trend. On both datasets, performance is low for small values of α (e.g., 0.1-0.4), as the augmentation is too weak to provide a sufficient regularization effect. As α increases, performance rises steadily, peaking in the range of 0.6 to 0.8 before plateauing or slightly decreasing. For instance, on CS, the accuracy climbs from 91.74% at $\alpha = 0.1$ to a peak of 92.74% at $\alpha = 0.4$, and remains high at 93.40% for α values of 0.6 and 0.8. This demonstrates that MESSAGESHIFT is not highly sensitive to the precise choice of α ,

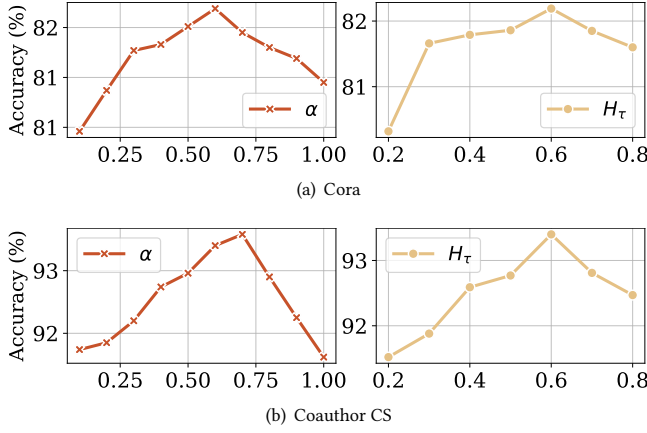


Figure 2: Hyperparameter Sensitivity Analysis.

making it easy to tune. An overly large value ($\alpha > 0.9$) can begin to introduce too much noise, leading to a minor drop in performance, which is expected behavior for a data augmentation method.

4.4.2 Sensitivity to Entropy Threshold H_τ . The entropy threshold H_τ is critical for the Selective Targeting (ST) strategy, determining the cutoff for which nodes are considered “uncertain” and thus targeted for augmentation. The analysis reveals that this parameter also behaves predictably. When the threshold is very low (e.g., 0.1–0.2), performance is suboptimal because too few nodes are being augmented, limiting the strategy’s impact. As H_τ increases, more nodes in uncertain regions are included, and performance rises to a peak around the median entropy level (a threshold of 0.5–0.6). On Cora, the accuracy increases from 80.96% at $H_\tau = 0.1$ to a peak of 82.19% at $H_\tau = 0.6$. If the threshold is set too high (e.g., $H_\tau > 0.8$), the strategy becomes less “selective,” and performance gradually declines as the augmentation starts to be applied to more stable, low-entropy regions where it is less needed. This confirms that the ST strategy is effective and that a simple heuristic, such as choosing the median entropy, provides a strong and robust default.

4.5 Q4: Representation Quality Analysis

To intuitively assess how MESSAGESHIFT improves representation quality, we visualize learned node embeddings using t-SNE. We train a GCN on Cora under an extreme low-label setting (5 labeled nodes per class) and project the output embeddings of four models to 2D, as shown in Fig. 3.

In Fig. 3(a), the embeddings from the Vanilla GCN show significant class overlap, indicating that the model fails to learn a discriminative representation space without sufficient regularization. The clusters are poorly defined, and nodes from different classes are heavily intermingled. Fig. 3(b) shows that applying DROPEdge, a standard structural augmentation method, offers only marginal improvements. While some clusters appear slightly more cohesive, the overall class separation remains poor, highlighting the limitations of context-agnostic perturbations.

In contrast, our methods yield more structured embeddings. As shown in Fig. 3(c), MESSAGESHIFT_{Base} already forms more distinct

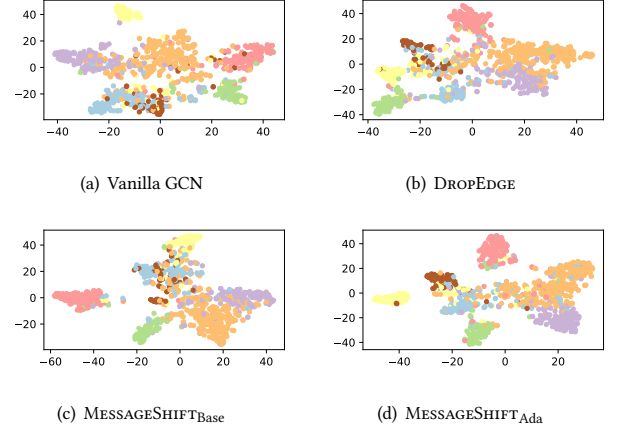


Figure 3: t-SNE Visualization of Node Embeddings.

clusters with lower intra-class variance, supporting the effectiveness of message-level augmentation. Fig. 3(d) further shows that MESSAGESHIFT_{Ada} produces the most compact and well-separated clusters. Notably, the previously scattered brown class becomes a tight, distinct group. Overall, these results indicate that our adaptive strategy improves the discriminability of the learned embedding space, beyond boosting classification accuracy.

5 Conclusion

In this work, we identify a fundamental limitation in existing graph data augmentation techniques: their indirect and often context-agnostic approach to regularizing GNNs, which typically involves perturbing the static inputs (graph structure or node features) rather than the dynamic information flow itself. To address this, we propose MESSAGESHIFT, a new data augmentation paradigm that operates directly at the fine-grained level of the message-passing mechanism. Our core innovation is a context-aware “shift” operation that perturbs each message based on its own local neighborhood information, allowing for both message smoothing and sharpening. This reframes graph augmentation from a static data pre-processing step to a dynamic, computational regularization strategy. Furthermore, we enhance this core mechanism with a suite of three adaptive strategies that intelligently control where, how, and when to apply augmentation. We believe MESSAGESHIFT provides a promising and principled direction for developing more robust and powerful graph representation learning models.

6 Acknowledgements

The authors gratefully acknowledge the support from The GREAT Smart City Institute at The Hong Kong University of Science and Technology. This work was supported in part by the National Natural Science Foundation of China under Grants 62425605, 62133012, and 62303366, and in part by the Key Research and Development Program of Shaanxi under Grants 2025CY-YBXM-041, 2022ZDLGY01-10, and 2024CY2-GJHX-15.

References

- [1] Yassine ABBAHADDOU, Fragkiskos D. Malliaros, Johannes F. Lutzeyer, Amine M. Aboussalah, and Michalis Vazirgiannis. 2025. Gaussian Mixture Models Based Augmentation Enhances GNN Generalization. <https://openreview.net/forum?id=0Th6bCZwKt>
- [2] Yassine Abbahaddou, Fragkiskos D. Malliaros, Johannes F. Lutzeyer, Amine Mohamed Aboussalah, and Michalis Vazirgiannis. 2025. Graph Neural Network Generalization with Gaussian Mixture Model Based Augmentation. *arXiv:2411.08638 [cs.LG]* <https://arxiv.org/abs/2411.08638>
- [3] Sami Abu-El-Haija, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, and Aram Galstyan. 2019. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *international conference on machine learning*. PMLR, 21–29.
- [4] Tobias Bernecker, Ghalia Rehawi, Janine Knauer-Arloth, and Annalisa Marsico. 2024. Random Walk Diffusion For Graph Generation. <https://openreview.net/forum?id=IL9o1meezQ>
- [5] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and deep graph convolutional networks. In *International conference on machine learning*. PMLR, 1725–1735.
- [6] Zhaoliang Chen, Zhihao Wu, Ylli Sadikaj, Claudia Plant, Hong-Ning Dai, Shipping Wang, Yiu-Ming Cheung, and Wenzhong Guo. 2025. Adedgedrop: Adversarial edge dropping for robust graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [7] Donato Crisostomi, Simone Antonelli, Valentino Maiorca, Luca Moschella, Riccardo Marin, and Emanuele Rodolà. 2022. Metric based few-shot graph classification. In *Learning on Graphs Conference*. PMLR, 33–1.
- [8] Donato Crisostomi, Simone Antonelli, Valentino Maiorca, Luca Moschella, Riccardo Marin, and Emanuele Rodolà. 2022. Metric Based Few-Shot Graph Classification. *arXiv preprint arXiv:2206.03695* (2022).
- [9] Huaming Du, Long Shi, Xingyan Chen, Yu Zhao, Hegui Zhang, Carl Yang, Fuzhen Zhuang, and Gang Kou. 2024. Representation Learning of Temporal Graphs with Structural Roles. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 654–665.
- [10] Taoran Fang, Zhiqing Xiao, Chunping Wang, Jiarong Xu, Xuan Yang, and Yang Yang. 2023. Dropmessage: Unifying random dropping for graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 4267–4275.
- [11] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph random neural networks for semi-supervised learning on graphs. *Advances in neural information processing systems* 33 (2020), 22092–22103.
- [12] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [13] Hongyu Guo and Yongyi Mao. 2021. ifMixup: Interpolating Graph Pair to Regularize Graph Classification. *arXiv e-prints* (2021), arXiv–2110.
- [14] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in neural information processing systems*. 1024–1034.
- [15] Xiaotian Han, Zhimeng Jiang, Ninghao Liu, and Xia Hu. 2022. G-mixup: Graph data augmentation for graph classification. In *International Conference on Machine Learning*. PMLR, 8230–8248.
- [16] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *International conference on machine learning*. PMLR, 4116–4126.
- [17] Jongwon Jeong, Hoyeop Lee, Hyui Geon Yoon, Beomyoung Lee, Junhee Heo, Geonsoo Kim, and Kim Jin Seon. 2024. iGraphMix: Input Graph Mixup Method for Node Classification. In *The Twelfth International Conference on Learning Representations*.
- [18] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [19] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997* (2018).
- [20] Dooho Lee, Myeong Kong, Sagad Hamid, Cheonwoo Lee, and Jaemin Yoo. 2025. Aggregation Buffer: Revisiting DropEdge with a New Parameter Block. *arXiv preprint arXiv:2505.20840* (2025).
- [21] Weigang Lu, Ziyu Guan, Wei Zhao, Yaming Yang, and Long Jin. 2024. Nodemixup: Tackling under-reaching for graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 14175–14183.
- [22] Weigang Lu, Ziyu Guan, Wei Zhao, Yaming Yang, Yuanhai Lv, Lining Xing, Baosheng Yu, and Dacheng Tao. 2023. Pseudo contrastive learning for graph-based semi-supervised learning. *arXiv preprint arXiv:2302.09532* (2023).
- [23] Weigang Lu, Ziyu Guan, Wei Zhao, Yaming Yang, Yibing Zhan, Yiheng Lu, and Dapeng Tao. 2025. AGMixup: Adaptive Graph Mixup for Semi-supervised Node Classification. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 18 (Apr. 2025), 19143–19151. doi:10.1609/aaai.v39i18.34107
- [24] Weigang Lu, Yibing Zhan, Binbin Lin, Ziyu Guan, Liu Liu, Baosheng Yu, Wei Zhao, Yaming Yang, and Dacheng Tao. 2024. SkipNode: On Alleviating Performance Degradation for Deep Graph Convolutional Networks. *IEEE Transactions on Knowledge and Data Engineering* (2024), 1–14. doi:10.1109/TKDE.2024.3374701
- [25] Xinyu Ma, Xu Chu, Yasha Wang, Yang Lin, Junfeng Zhao, Liantao Ma, and Wenwu Zhu. 2024. Fused Gromov-Wasserstein Graph Mixup for Graph-level Classifications. *Advances in Neural Information Processing Systems* 36 (2024).
- [26] Madeline Navarro and Santiago Segarra. 2022. GraphMAD: Graph Mixup for Data Augmentation using Data-Driven Convex Clustering. *arXiv preprint arXiv:2210.15721* (2022).
- [27] Joonhyung Park, Hajin Shim, and Eunho Yang. 2022. Graph transplant: Node saliency-guided graph mixup with local structure preservation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 7966–7974.
- [28] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2019. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903* (2019).
- [29] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI magazine* 29, 3 (2008), 93–93.
- [30] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [31] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [32] Vikas Verma, Meng Qu, Kenji Kawaguchi, Alex Lamb, Yoshua Bengio, Juho Kannala, and Jian Tang. 2021. Graphmix: Improved training of gnns for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 10024–10032.
- [33] Yiwei Wang, Wei Wang, Yuxuan Liang, Yujun Cai, and Bryan Hooi. 2021. Mixup for node and graph classification. In *Proceedings of the Web Conference 2021*. 3663–3674.
- [34] Jong whi Lee and Jinhong Jung. 2023. Time-aware Random Walk Diffusion to Improve Dynamic Graph Learning. *arXiv:2211.01214 [cs.LG]* <https://arxiv.org/abs/2211.01214>
- [35] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *International conference on machine learning*. PMLR, 6861–6871.
- [36] Lirong Wu, Haitao Lin, Zhangyang Gao, Cheng Tan, Stan Li, et al. 2021. Graphmixup: Improving class-imbalanced node classification on graphs by self-supervised context prediction. *arXiv preprint arXiv:2106.11133* (2021).
- [37] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).
- [38] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. In *International conference on machine learning*. PMLR, 5453–5462.
- [39] Yaming Yang, Ziyu Guan, Wei Zhao, Weigang Lu, and Bo Zong. 2022. Graph substructure assembling network with soft sequence and context attention. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4894–4907.
- [40] Yaming Yang, Ziyu Guan, Wei Zhao, Weigang Lu, and Bo Zong. 2022. Graph substructure assembling network with soft sequence and context attention. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4894–4907.
- [41] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [42] Lingxiao Zhao and Leman Akoglu. 2019. Pairnorm: Tackling oversmoothing in gnns. *arXiv preprint arXiv:1909.12223* (2019).
- [43] Tong Zhao, Yozen Liu, Leonardo Neves, Oliver Woodford, Meng Jiang, and Neil Shah. 2021. Data augmentation for graph neural networks. In *Proceedings of the aaii conference on artificial intelligence*, Vol. 35. 11015–11023.